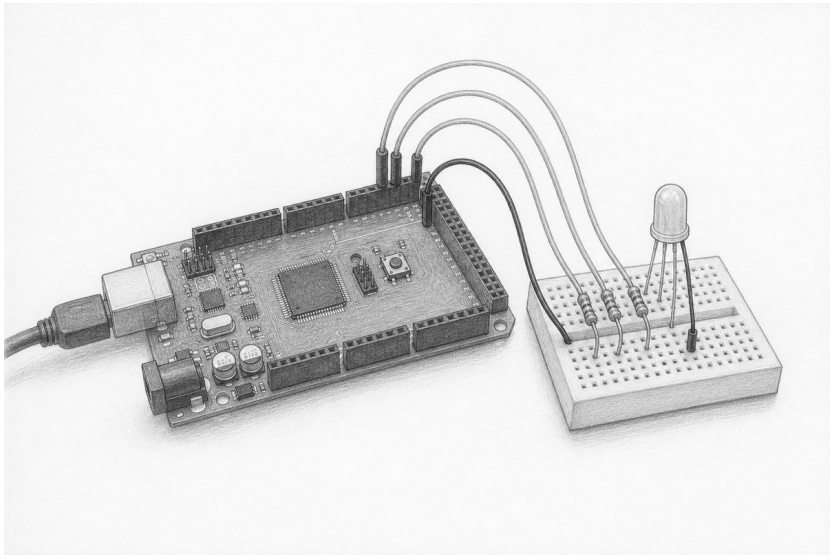
Compose three controlled currents

Lesson 004: `PwmOutput` and `RgbLed`

Calculate → wire → test → mix



Orientation plate. Use the graphite view to recognize the parts and their approximate placement. It is not a pin map. Breadboard connectivity and RGB LED leg order vary; the exact circuit and your component's datasheet govern the build.

Question	Can three owning PWM endpoints safely compose one deterministic common-cathode RGB LED?
Time	70–95 minutes
Prerequisites	Lessons 001–003; explicit status handling; resistor reading
You need	Mega 2560, USB cable, breadboard, common-cathode RGB LED, three verified resistors, four jumpers, and a multimeter
Evidence	Current worksheet, exact host trace, color observations, and high-impedance teardown
Safety class	E1: USB-powered Mega and current-limited LEDs only
Status	Host-verified and experimental; not yet hardware-accepted or stable

Safety boundary

Use only USB power and the low-energy circuit shown here. Remove every power source before changing a wire. Never omit or share the three current-limiting resistors. Confirm the LED is *common cathode*; a common-anode part needs different semantics and is outside this lesson.

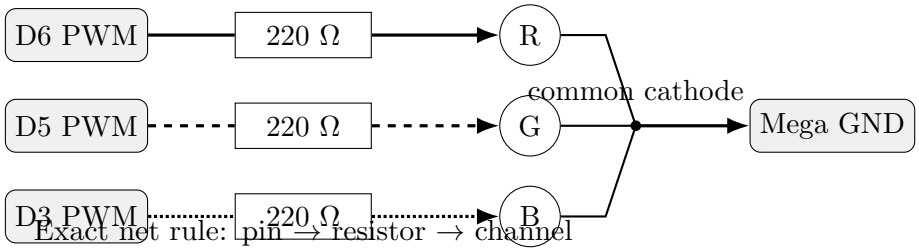
Stop for heat, odor, smoke, unexpected USB disconnects, an unknown resistor value, or an unidentified LED common lead. Disconnect USB upstream. Do not troubleshoot a powered breadboard by moving jumpers.

Check — before wiring

☐ USB removed ☐ LED datasheet or verified pinout ☐ three resistors measured ☐ rails traced

1. Read the exact common-cathode circuit

The cathodes of the three LED dies meet at one common lead connected to Mega `GND`. Each anode has its *own* series resistor. Pins 6, 5, and 3 are PWM-capable on the Mega 2560.



Exact net rule: pin → resistor → channel anode; the one common cathode → GND.

Check — continuity check while unpowered

☐ no pin-to-pin short ☐ each channel has one resistor ☐ common lead reaches GND only

2. Budget current before applying power

Use the LED datasheet's forward voltage V_f, not lens color or guesswork. For a first estimate,

$$I = \frac{V_{\text{pin}} - V_f}{R}.$$

With $V_{\text{pin}} = 5.0$ V and $R = 220\,\Omega$, a red die at $V_f = 2.0$ V estimates 13.6 mA; a green or blue die at 3.0 V estimates 9.1 mA. All three at full duty estimate 31.8 mA total. These are estimates, not guarantees: output voltage falls under load and device limits are simultaneous constraints.

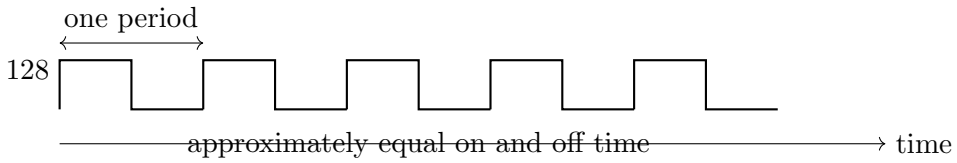
Repeat the calculation with the datasheet's *minimum* V_f, which produces the largest current. Sum all three channels at duty 255. Do not apply power unless each channel and the total remain below the documented operating limits with margin.

Channel	Measured R	Datasheet V_f	Estimated I	Accepted?
red				
green				
blue				
Total at full duty				

Do not design to the microcontroller's absolute maximum. Check the official ATmega2560 and LED datasheets for per-pin, grouped-pin, total-device, and LED limits. If any estimate lacks margin, increase resistance or stop.

3. Understand duty, not "analog voltage"

`PwmOutput::Duty` is an 8-bit value: 0 is 0% high duty, 255 is 100% high duty, and intermediate values switch rapidly. Duty 128 is about 50%, but it does not command a steady 2.5 V. In this lesson's common-cathode LED, zero is inactive. Other circuits may assign different semantics.



Predict. Order the apparent brightness for duties 0, 16, 64, 128, and 255:

---

4. Read the exact endpoint API

`PwmOutput` claims one PWM-capable Mega pin. Initialization writes the initial duty. Shutdown writes zero, selects input mode, and releases the claim. All operations are exception-free; destruction performs idempotent shutdown.

```
adk::Runtime   runtime;
adk::PwmOutput red (runtime.resources (), 6, 0);

adk::Status status = red.initialize ();
if (status == adk::Status::Ok)
{
    status = red.write (128);
}

red.shutdown ();
```

An invalid pin reports `InvalidPin`; a valid non-PWM pin reports `Unsupported`; a claimed pin reports `ResourceBusy`; writing before initialization reports `NotInitialized`.

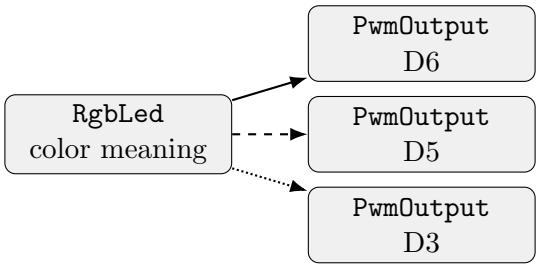
5. Compose the semantic RGB component

`RgbLed` owns three `PwmOutput` objects. Its channel descriptors retain both pin identity and the documented resistor value. For a common-cathode LED, color byte values map directly to duty.

```
adk::Runtime runtime;
adk::RgbLed  led (runtime.resources (),
                  {6, 220},
                  {5, 220},
                  {3, 220});

if (led.initialize () == adk::Status::Ok)
{
    led.set (adk::Rgb (255, 64, 0));
    led.off ();
}
```

Initialization is transactional: red, green, and blue claims must all succeed. A later failure releases earlier claims. `set()` updates all three channels; `off()` sets `Rgb(0,0,0)`. Shutdown proceeds through the owned endpoints, leaving every channel at zero and high impedance.



6. Prove behavior on the host

The fake Arduino trace is the electrical contract's executable evidence. Write tests for:

1. every Mega PWM pin and representative valid non-PWM pins;
2. duty endpoints 0 and 255 plus one intermediate value;
3. write before initialization and repeated initialization;
4. duplicate claims, release, and reuse;
5. destruction: `analogWrite(pin, 0)` before `pinMode(pin, INPUT)`;
6. RGB initialization failure on the second and third channel;
7. exact red–green–blue write order for several colors;
8. repeated `off()` and `shutdown()`;
9. two identical runs producing identical traces.

Trace exercise. Predict the complete operation trace for `RgbLed::initialize()`, `set(Rgb(255,64,0))`, and destruction:

7. Build and observe

Run the repository's host checks and compile the canonical Mega example before upload. Inspect every status; an ignored failure makes observations ambiguous.

```
make check
make arduino
arduino-cli board list
```

1. Remove USB. Build and continuity-check the exact circuit.
2. Connect USB only. Upload to the port you explicitly identified.
3. Command red, green, blue, white, off, and one mixed color.

4. Record observed color and whether any channel appears swapped.
5. Run the shutdown phase. Confirm all channels are inactive.
6. Remove USB before correcting any channel identity.

Command	Expected	Observed	Trace/status
`Rgb(255,0,0)`	red		
`Rgb(0,255,0)`	green		
`Rgb(0,0,255)`	blue		
`Rgb(255,255,255)`	combined white		
`Rgb(0,0,0)`	off		

8. Explain what the abstraction prevents

1. Why must each die have a separate resistor?
2. Why does common cathode permit direct duty semantics?
3. Why should `RgbLed` compose rather than inherit from `PwmOutput`?
4. What must happen if the blue pin is already claimed?
5. Why is a stored resistor value useful even though software cannot enforce the physical resistor?
6. Which evidence proves teardown better than visual darkness?

Exit ticket

☐ circuit checked unpowered ☐ current budget recorded ☐ host tests pass ☐ Mega build passes ☐ status paths handled ☐ teardown trace verified

Companion material. Use the HTML lesson for exact source links, current API signatures, errata, Mega pin-capability tables, and authoritative datasheet links. Use this PDF for bench predictions, wiring checks, calculations, observations, and sign-off. This lesson remains host-verified until its physical acceptance record is published.

Arduino Mega 2560 documentation Microchip ATmega2560 datasheet